NVIDIA

SDET Pre-Internship Self-Study Program

Deep Learning Infrastructure Track

March 5 → May 18, 2025 | 10.5 Weeks | 6 Phases

Ph1:Git Ph2:Docker Ph3:K8s Ph4:Inference Ph5:TRT-LLM Ph6:NeMo

Beginner-friendly · Application-first · One project, six technologies

Self-Directed — No GPU required until Phase 5

How to Use This Guide

This guide is your complete roadmap from today (March 5) to your first day at NVIDIA on May 18. It is not a list of topics to read about. Every phase produces something — code, tests, a Docker image, a running Kubernetes pod — because application is the only thing that actually builds the muscle memory you will need on day one. Reading about Docker is not the same as debugging a broken Dockerfile at 11pm.

📌 **The Golden Rule** Never move to the next phase until you hit the checkpoint of the current one. A shaky foundation in Docker will make Kubernetes confusing. A shaky foundation in Kubernetes will make TRT-LLM debugging feel impossible. Slow is smooth, smooth is fast.

🎓 **On Your Parallel Classes** You are taking Python and ML classes simultaneously. This guide does not re-teach those — it gives you applied projects that USE what you learn in class. When your class covers functions, you will be writing functions in your Docker project. When your class covers neural networks, you will be benchmarking one. Let the two reinforce each other.

Your Timeline at a Glance

Phase	Dates	Topic	GPU Required?	Key Deliverable
1	Mar 5–12	Git Workflow	No	inference-sandbox repo + first MR
2	Mar 12–Apr 2	Docker	No	Containerised app + Compose stack
3	Apr 2–16	Kubernetes	No	App deployed on minikube
4	Apr 16–30	AI Inference Concepts	Free T4	Benchmark script + quant test
5	Apr 30–May 14	TRT-LLM Hands-On	Paid GPU	Built engine + correctness test
6	May 14–18	NeMo + Dynamo Orientation	No	Architecture notes + vocab

The Throughline Project: inference-sandbox

Rather than disconnected tutorials, you will build one project across all six phases. It starts as a single Python script and grows — week by week — into a containerised, Kubernetes-deployed, automated-tested AI inference benchmarking system. By Phase 5, it will run a real TRT-LLM engine. By the time you walk into NVIDIA, you will have something concrete to talk about in every conversation.

Phase 1: A Python benchmarking script in a GitLab repo with proper branch/MR workflow

Phase 2: The script containerised in a multi-stage Docker image, pushed to a registry

Phase 3: The container deployed on a local Kubernetes cluster with a Service in front of it

Phase 4: The script benchmarks a real HuggingFace model — TTFT, throughput, quantisation tests

Phase 5: The script benchmarks a TRT-LLM engine and runs a correctness regression test

Phase 6: Architecture notes doc added — explains how NeMo and Dynamo would extend the system

📁 **Setup Now** Create a free account at gitlab.com (not GitHub — NVIDIA uses GitLab). Create a new project called inference-sandbox. Set the default branch to main. Clone it to your machine. This is the only setup you need for Phase 1.

PHASE 1 GIT WORKFLOW

March 5 – March 12 | *Learn the professional development workflow — branches, MRs, CI — using your own project*

Why This Phase Exists

You said you have "a bit of GitHub experience." That probably means you know add/commit/push. What you likely have NOT done is work in a team workflow: feature branches, protected main, merge requests with descriptions, and automated checks that run before a merge is allowed. That is exactly how NVIDIA operates. Every single line of code that goes into a production inference system goes through a merge request. This phase builds that habit from day one so it is automatic by the time you arrive.

Core Concepts

The Branch-MR-Merge Cycle

This is the atomic unit of professional software development. Every change — no matter how small — follows this exact cycle:

1. Pull latest main: git pull origin main
2. Create a feature branch: git checkout -b feature/your-change-name
3. Make your changes. Commit frequently with descriptive messages.
4. Push the branch: git push origin feature/your-change-name
5. Open a Merge Request on GitLab. Write a description explaining WHAT changed and WHY.
6. CI pipeline runs automatically. Fix anything it flags.
7. Merge into main. Delete the feature branch.

⚠️ **The Habit That Matters Most** Never commit directly to main. Not once. Not even for a one-line change. The discipline feels unnecessary when you are working alone — that is exactly when you build the habit. If you skip it now, you will accidentally do it at NVIDIA and trigger a rollback incident on your second week.

Commit Messages

A good commit message answers: what changed, and why. The standard at most companies is the Conventional Commits format:

```
# Format: <type>(<scope>): <short description>
#
# type: feat | fix | test | docs | refactor | chore
# scope: optional, the component you changed
# description: present tense, lowercase, no period

# Good examples:
feat(benchmark): add TTFT measurement to timing loop
```

```
fix(docker): correct CUDA base image tag from 12.3 to 12.4
test(inference): add perplexity assertion for FP16 baseline
docs(readme): add local setup instructions for minikube

# Bad examples (never do these):
"update"
"fix stuff"
"WIP"
"asdfasdf"
```

GitLab CI/CD — Your Automated Safety Net

A .gitlab-ci.yml file in your repo root tells GitLab what to run every time you push. Think of it as a robot reviewer that checks your code before it can merge. In Phase 1 your pipeline will be simple — just run your Python file. By Phase 5 it will build Docker images, run TRT-LLM tests, and post benchmark reports.

```
# .gitlab-ci.yml — your starting pipeline (paste this into your repo)

stages:
  - lint
  - test

lint:python:
  stage: lint
  image: python:3.11-slim
  script:
    - pip install flake8 --quiet
    - flake8 benchmark.py --max-line-length=100
  only:
    - merge_requests
    - main

test:run:
  stage: test
  image: python:3.11-slim
  script:
    - python benchmark.py --self-test
  only:
    - merge_requests
    - main
```

💡 **Reading CI Output** When your pipeline fails, click the red X next to the job name, not the MR. The job log shows the exact line that failed. 90% of the time it is a missing package, a typo in a file path, or a flake8 whitespace violation. Read the last 20 lines of the log before asking for help.

Protected Branches

Go to your GitLab project → Settings → Repository → Protected branches. Set main to "Allowed to push: No one" and "Allowed to merge: Maintainers." Then add yourself as a Maintainer in Members. Now you CANNOT push directly to main — even you. The only way to change main is through an MR. This mirrors NVIDIA's setup exactly.

Phase 1 Projects

PROJECT 1.1 BUILD THE BENCHMARK.PY FOUNDATION

Objective: Create the core Python file your entire project will grow from.

1. Create benchmark.py in your repo root.
2. Write a function time_reverse(text: str) -> dict that reverses the input string and returns {"input_length": int, "output": str, "duration_ms": float}.
3. Write a main() block that calls time_reverse() with three different inputs (short, medium, long strings) and prints a formatted table of results.
4. Add a --self-test flag (use argparse) that runs 3 assertions: output is correct, duration_ms is a positive float, input_length is correct.
5. Add a docstring to every function. Flake8 must pass with zero warnings.
6. Commit this on a branch named feature/benchmark-foundation. Open an MR. Write a real description. Merge it.

✓ **Done when:** CI pipeline passes on the MR. Output table prints cleanly. --self-test exits 0.

💡 Your --self-test flag is your first real automated test. It is also how your CI job verifies the script works. This pattern — ship testable code — is the SDET mindset applied from your very first file.

PROJECT 1.2 ADD A RESULTS LOGGER

Objective: Extend benchmark.py to persist results to a JSON file — your first step toward a real test harness.

1. Add a function save_results(results: list, path: str) that appends benchmark results to a JSON Lines file (one JSON object per line).
2. Add a function load_results(path: str) -> list that reads the file back.
3. Write an assertion in --self-test: save 3 results, load them back, assert the loaded list matches what was saved.
4. Add a .gitignore that excludes *.jsonl result files (you want code, not data, in git).
5. Branch: feature/results-logger. Commit in two separate commits: one for save_results, one for load_results. Practice atomic commits.

✓ **Done when:** MR shows two clean commits. --self-test passes. .gitignore prevents .jsonl files from appearing in git status.

💡 Committing save and load separately makes your git history readable. If load_results has a bug next month, git log shows you exactly which commit introduced it.

PROJECT 1.3 WRITE A REAL .GITLAB-CI.YML

Objective: Set up your CI pipeline so every MR is automatically validated.

1. Paste the starter .gitlab-ci.yml from the Core Concepts section into your repo.
2. Modify test:run to call python benchmark.py --self-test.
3. Add a third job test:import that simply runs python -c "import benchmark" to catch import-time errors early.
4. Deliberately introduce a flake8 violation (trailing whitespace). Push it. Watch the lint job fail. Fix it. Push again. Watch it pass.
5. Enable "Pipelines must succeed" in MR settings so you cannot merge a failing MR.

✓ **Done when:** GitLab shows green checkmarks on your MR. Deliberately broken code causes a red X. The merge button is greyed out until CI passes.

💡 Breaking things intentionally and watching the system catch it is how you build confidence in your test infrastructure. If you never see it fail, you do not know it works.

PROJECT 1.4 WRITE YOUR FIRST LEARNING_LOG.MD ENTRY

Objective: Start the weekly reflection habit that will impress your manager on day one.

1. Create LEARNING_LOG.md in your repo root.
2. Write a Phase 1 entry using the template from Appendix C.
3. Specifically answer: What does a Merge Request do that a direct push does not? What is a CI pipeline runner? Why does commit message format matter?
4. Commit this on main via an MR (yes, even for a markdown file).

✓ **Done when:** File exists in repo, answers are in your own words — not copied from this guide.

💡 If you cannot explain something in writing, you do not actually understand it yet. The log is not busywork — it is a forcing function for real comprehension.

Phase 1 Checkpoint

✅ **Before Moving to Phase 2** You should be able to: (1) explain what a merge request is for without looking it up, (2) write a .gitlab-ci.yml with two jobs from memory, (3) explain why protected branches exist, (4) describe what "atomic commits" means and why it matters. If any of these feel shaky, spend another day on Phase 1. Phase 2 will still be there.

PHASE 2 DOCKER

March 12 – April 2 (3 weeks) | *Containerise your inference-sandbox app — the most critical phase of your prep*

Why Docker Is Your Most Important Phase

Everything at NVIDIA runs in containers. TRT-LLM is distributed as a container. NeMo training jobs run in containers. Kubernetes orchestrates containers. The Dynamo inference stack is containers talking to containers. If Docker feels uncertain, every single downstream technology will be harder. Three weeks here is not excessive — it is exactly right.

Mental Model: Images vs Containers

Before reading anything else, lock in this distinction:

```
# An IMAGE is a blueprint. A CONTAINER is a running instance of that blueprint.
#
# Analogy:
#   Image     = a recipe
#   Container = the meal you cooked from that recipe
#
# You can cook (run) the same recipe (image) many times.
# Each meal (container) is independent — changes to one don't affect others.
# The recipe (image) itself never changes once built.
#
# docker build  →  creates an image from a Dockerfile
# docker run    →  creates and starts a container from an image
# docker ps     →  lists running containers
# docker images →  lists images on your machine
```

Core Concepts

Layers and the Build Cache

A Docker image is made of stacked layers. Each instruction in your Dockerfile (FROM, RUN, COPY, etc.) creates a new layer. Layers are cached — if a layer has not changed, Docker reuses the cache instead of rebuilding it. This is why instruction ORDER matters enormously:

```
# ❌ SLOW Dockerfile — cache breaks every time you change benchmark.py
FROM python:3.11-slim
COPY . /app           # Changes on every edit → invalidates everything below
RUN pip install -r requirements.txt   # Re-runs every single time

# ✅ FAST Dockerfile — requirements cached unless requirements.txt changes
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .       # Only changes when deps change
RUN pip install -r requirements.txt   # Only re-runs when deps change
COPY . /app                   # Changes on every edit — but it's the last step
CMD ["python", "benchmark.py"]
```

💡 **The Rule** Always COPY what changes LEAST first, and what changes MOST last. requirements.txt changes rarely. Your source code changes constantly. This single rule will save you hours of rebuild time during your internship.

Multi-Stage Builds

A multi-stage Dockerfile uses multiple FROM instructions. Early stages do heavy build work (compiling, installing dev tools). The final stage copies only the runtime artifacts — keeping your final image lean. This is mandatory for GPU containers at NVIDIA because a devel CUDA image is ~8GB but a runtime image is ~3GB.

```
# Multi-stage: builder stage does the heavy lifting
FROM python:3.11 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Final stage: only what is needed to run
FROM python:3.11-slim
WORKDIR /app
# Copy only installed packages from builder — not the entire Python install
COPY --from=builder /root/.local /root/.local
COPY benchmark.py .
ENV PATH=/root/.local/bin:$PATH
CMD ["python", "benchmark.py"]
```

NVIDIA Base Images

For GPU work, you start from NVIDIA's official CUDA images on Docker Hub / NGC. The tag format encodes everything you need to know:

```
# nvidia/cuda:<cuda_version>-<variant>-<os>
#
# Variants (in order of increasing size):
#   base    — CUDA runtime only (~200MB). For apps that just need CUDA at
runtime.
#   runtime — CUDA + cuDNN runtime (~1.5GB). For running pre-compiled DL models.
#   devel   — CUDA + cuDNN + full compiler toolchain (~4GB). For compiling from
source.
#
# Examples:
FROM nvidia/cuda:12.4.1-base-ubuntu22.04         # Smallest. Just CUDA runtime.
FROM nvidia/cuda:12.4.1-cudnn9-runtime-ubuntu22.04   # TRT-LLM inference base.
FROM nvidia/cuda:12.4.1-cudnn9-devel-ubuntu22.04     # Build TRT from source.
#
# Rule: use the SMALLEST variant that satisfies your runtime needs.
# You can read these on hub.docker.com/r/nvidia/cuda even without a GPU.
```

📌 **You Cannot Run GPU Images Without a GPU** You can BUILD and INSPECT nvidia/cuda images on your laptop — Docker will pull and cache them fine. But docker run will fail unless you have an NVIDIA GPU +

Container Toolkit installed. Study the tags and Dockerfiles now; you will run them for real in Phase 5 on a cloud GPU.

Docker Compose

Compose lets you define and run multi-container applications from a single YAML file. Instead of typing long docker run commands with all their flags, you declare the entire stack once and run docker compose up. This is how local development environments and test fixtures are managed across the entire industry.

```
# docker-compose.yml — a two-service stack
services:
  app:
    build: .                          # Build image from local Dockerfile
    ports:
      - "8080:8080"                   # host:container port mapping
    environment:
      - REDIS_HOST=redis              # 'redis' resolves to the redis container
      - LOG_LEVEL=DEBUG
    volumes:
      - ./results:/app/results        # Bind mount: host directory into container
    depends_on:
      - redis                         # Don't start app until redis is up

  redis:
    image: redis:7-alpine             # Pull from Docker Hub, no build needed
    ports:
      - "6379:6379"

# Usage:
#   docker compose up          start everything
#   docker compose up --build  rebuild images first
#   docker compose down        stop and remove containers
#   docker compose logs app    stream logs from the app service
```

Phase 2 Projects

PROJECT 2.1 CONTAINERISE BENCHMARK.PY

Objective: Package your Python script into a Docker image with proper layering.

1. Install Docker Desktop on your machine (docs.docker.com/get-docker).
2. Create a requirements.txt in your repo (just one line: flake8 for now — you will add more packages in later phases).
3. Write a Dockerfile that: uses python:3.11-slim as base, sets a WORKDIR, copies requirements.txt first, installs it, then copies benchmark.py.
4. Build it: docker build -t inference-sandbox:v1 .
5. Run it: docker run --rm inference-sandbox:v1 — verify your benchmark output prints.
6. Run it with self-test: docker run --rm inference-sandbox:v1 python benchmark.py --self-test
7. Add the Dockerfile to your repo via an MR. CI should still pass.

✓ **Done when:** docker run --rm inference-sandbox:v1 prints the benchmark table. --self-test exits with code 0. docker images shows your image under 200MB.

💡 Run docker images after your build. If the image is bigger than you expect, run docker history inference-sandbox:v1 to see exactly which layer added the size.

PROJECT 2.2 PROVE YOU UNDERSTAND LAYER CACHING

Objective: Build the image twice and explain exactly what happened — the critical mental model check.

1. Build your image fresh: docker build -t inference-sandbox:v1 . (note the time taken)
2. Change a single line in benchmark.py (add a comment). Rebuild: docker build -t inference-sandbox:v1 . (note which layers say "CACHED")
3. Now change requirements.txt (add a blank line). Rebuild again. Note which layers are no longer cached.
4. In LEARNING_LOG.md, write a section explaining: which layers were cached in step 2 vs step 3, WHY the cache broke where it did, and what this tells you about Dockerfile instruction ordering.

✓ **Done when:** Your LEARNING_LOG.md accurately explains the caching behaviour in your own words. Not copied — described from what you observed.

💡 This is not a typing exercise. If your explanation is wrong, the mental model is wrong. Re-read the Layers section and rebuild until your prediction matches the output.

PROJECT 2.3 CONVERT TO A MULTI-STAGE BUILD

Objective: Reduce your image size using a multi-stage Dockerfile.

1. Rewrite your Dockerfile as a two-stage build: a builder stage using python:3.11 (full) and a runtime stage using python:3.11-slim.
2. In the builder stage, install requirements with pip install --user.
3. In the runtime stage, COPY --from=builder only the installed packages.
4. Build both the old (single-stage) and new (multi-stage) images. Run docker images and document the size difference in your Learning Log.
5. Add a label to your final image: LABEL version="2.0" maintainer="your-name"
6. Run docker inspect inference-sandbox:v2 | grep -A5 Labels to verify.

✓ **Done when:** Multi-stage image is smaller than single-stage. docker inspect shows your labels. Container still runs correctly.

💡 If multi-stage ends up LARGER, you copied too much from the builder. Use pip show <package> to find exactly where packages install to, then copy only that path.

PROJECT 2.4 ADD REDIS WITH DOCKER COMPOSE

Objective: Build a real multi-container local stack — your first distributed system.

1. Add redis to requirements.txt.
2. Modify benchmark.py: after each benchmark run, use the redis Python client to push the result JSON to a Redis list named benchmark:results. Read the REDIS_HOST environment variable (default to localhost if not set).
3. Add a --dump-results flag that reads all entries from Redis and prints them.
4. Write a docker-compose.yml with your app service and a redis:7-alpine service. Pass REDIS_HOST=redis as an environment variable to your app.
5. Run docker compose up. In another terminal: docker compose run app python benchmark.py --dump-results
6. Verify results from previous runs are persisted.

✓ **Done when:** docker compose up starts both containers. benchmark.py writes to Redis. --dump-results reads back persisted entries from previous runs.

💡 When your app cannot connect to Redis, 90% of the time it is a name resolution issue. Run docker compose exec app ping redis to confirm the containers can see each other. In Compose, service names ARE hostnames on the internal network.

PROJECT 2.5 PUSH TO GITLAB CONTAINER REGISTRY

Objective: Publish your image to a real registry — the final step of every Docker workflow.

1. In GitLab: Project → Packages & Registries → Container Registry. Note your registry URL (registry.gitlab.com/<username>/inference-sandbox).
2. Authenticate: docker login registry.gitlab.com
3. Tag your image: docker tag inference-sandbox:v2 registry.gitlab.com/<username>/inference-sandbox:v2
4. Push: docker push registry.gitlab.com/<username>/inference-sandbox:v2
5. Add a CI job to .gitlab-ci.yml that builds and pushes the image automatically on every MR. Use $CI_REGISTRY, $CI_REGISTRY_USER, $CI_REGISTRY_PASSWORD (these are built-in GitLab CI variables — no secrets to manage manually).

✓ **Done when:** Image visible in GitLab Container Registry UI. CI job builds and pushes on every MR push.

💡 Use $CI_COMMIT_SHORT_SHA as your image tag in CI so every commit produces a uniquely tagged image. This is production practice — you can always tell which code version an image was built from.

Phase 2 Checkpoint

✅ **Before Moving to Phase 3** You should be able to: (1) write a multi-stage Dockerfile from scratch, (2) explain why layer order matters for caching, (3) run a multi-container app with Docker Compose, (4) explain what devel vs runtime NVIDIA CUDA images are for, (5) push an image to a registry via CI. Every one of these will come up in your first week at NVIDIA.

PHASE 3 KUBERNETES

April 2 – April 16 (2 weeks) | *Deploy your containerised app to a real local cluster using minikube*

Why Kubernetes After Docker

Docker Compose runs your app on one machine. Kubernetes (K8s) runs it across many machines, restarts it if it crashes, scales it up when traffic spikes, and handles rolling upgrades without downtime. NVIDIA runs TRT-LLM inference services on GPU Kubernetes clusters — understanding the basics means you can read and write the deployment configs your team uses every day.

Setup — minikube

minikube runs a single-node Kubernetes cluster inside a VM or container on your laptop. It is the standard local K8s environment for learning and development.

```
# Install minikube (macOS with Homebrew):
brew install minikube kubectl

# Install minikube (Windows — use official installer from minikube.sigs.k8s.io)

# Start your cluster:
minikube start --driver=docker --memory=4096 --cpus=2

# Verify it's running:
kubectl cluster-info
kubectl get nodes          # Should show 1 node in Ready state

# Enable the dashboard (optional but useful for learning):
minikube dashboard         # Opens browser UI
```

Core Concepts

The Kubernetes Object Model

Everything in Kubernetes is an "object" described in YAML and submitted via kubectl apply. The four objects you need to know for this phase:

Pod: The smallest deployable unit — one or more containers that share a network namespace and can share storage. You rarely create Pods directly; you let a Deployment manage them.

Deployment: Declares "I want 2 running copies of this container at all times." Kubernetes continuously reconciles reality to match. If a Pod crashes, the Deployment creates a replacement automatically.

Service: A stable network endpoint in front of a set of Pods. Pods come and go (they get new IPs each time), but the Service IP is permanent. Clients always talk to the Service, never directly to a Pod.

ConfigMap: A Key-Value store for configuration (like REDIS_HOST=redis). Externalises config from your container image — the same image runs in dev, staging, and prod with different ConfigMaps.

Essential kubectl Commands

```
# Apply a YAML file (create or update the object)
kubectl apply -f deployment.yaml

# List objects
kubectl get pods                      # List pods in default namespace
kubectl get pods -n dl-inference      # List pods in a specific namespace
kubectl get deployments
kubectl get services

# Inspect an object (outputs detailed YAML + events — your main debugging tool)
kubectl describe pod <pod-name>
kubectl describe deployment inference-sandbox

# Stream logs from a running pod
kubectl logs <pod-name>               # Print logs
kubectl logs <pod-name> -f            # Follow (stream) logs

# Run a command inside a running container
kubectl exec -it <pod-name> -- /bin/bash      # Interactive shell
kubectl exec <pod-name> -- python benchmark.py --self-test

# Delete objects
kubectl delete -f deployment.yaml     # Delete what the YAML describes
kubectl delete pod <pod-name>         # Delete a specific pod (Deployment will
recreate it)

# Port-forward: expose a pod/service to your local machine
kubectl port-forward service/inference-sandbox 8080:8080
```

A Complete Deployment + Service

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inference-sandbox
  labels:
    app: inference-sandbox
spec:
  replicas: 2                          # Run 2 copies
  selector:
    matchLabels:
      app: inference-sandbox
  template:
    metadata:
      labels:
        app: inference-sandbox
    spec:
      containers:
      - name: benchmark
        image: registry.gitlab.com/<you>/inference-sandbox:v2
        command: ["python", "benchmark.py"]
        env:
        - name: REDIS_HOST
```

```
          valueFrom:
            configMapKeyRef:
              name: inference-config    # Reference a ConfigMap
              key: redis_host
        resources:
          requests:
            cpu: "250m"                 # 0.25 CPU cores
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "256Mi"
---
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: inference-sandbox
spec:
  selector:
    app: inference-sandbox              # Routes traffic to pods with this label
  ports:
  - port: 80
    targetPort: 8080
  type: ClusterIP                       # Internal only — use port-forward to
reach it locally
```

Phase 3 Projects

PROJECT 3.1 DEPLOY TO MINIKUBE

Objective: Get your inference-sandbox container running inside a real Kubernetes cluster.

1. Start minikube: minikube start
2. Point Docker at minikube's registry so you don't need to push to GitLab: eval $(minikube docker-env) then docker build -t inference-sandbox:v2 .
3. Create deployment.yaml and service.yaml based on the examples above. Use imagePullPolicy: Never so K8s uses your local image.
4. Apply them: kubectl apply -f deployment.yaml && kubectl apply -f service.yaml
5. Watch the pods start: kubectl get pods -w (the -w flag watches for changes)
6. Port-forward the service: kubectl port-forward service/inference-sandbox 8080:80
7. From another terminal: curl http://localhost:8080 — or run the benchmark via exec.

✓ **Done when:** kubectl get pods shows 2 Running pods. kubectl logs <pod> shows benchmark output. Port-forwarding lets you reach the service from your host machine.

💡 If a Pod is stuck in Pending or CrashLoopBackOff, run kubectl describe pod <pod-name>. The Events section at the bottom almost always explains exactly what went wrong — image not found, insufficient resources, etc.

PROJECT 3.2 EXTERNALISE CONFIG WITH A CONFIGMAP

Objective: Practice the K8s pattern for keeping configuration out of your container image.

1. Create configmap.yaml with key-value pairs: redis_host=redis, log_level=INFO.
2. Modify deployment.yaml to read REDIS_HOST and LOG_LEVEL from the ConfigMap (use the valueFrom: configMapKeyRef: pattern shown above).

3. Apply the ConfigMap and re-apply the Deployment.
4. kubectl exec into a running pod and run env | grep -E "REDIS|LOG" to verify the environment variables were injected.
5. Change the log_level in the ConfigMap to DEBUG, reapply, and verify the pod picks it up (you may need to restart pods: kubectl rollout restart deployment/inference-sandbox).

✓ **Done when:** kubectl exec into pod, env shows correct values from ConfigMap. Changing ConfigMap + restarting pods results in new values being picked up.

💡 ConfigMaps do not automatically restart pods when changed. This is intentional — you control when the new config takes effect. kubectl rollout restart is the clean way to do a zero-downtime config refresh.

PROJECT 3.3 OBSERVE POD FAILURE AND SELF-HEALING

Objective: Watch Kubernetes automatically recover from a pod crash — the most important K8s concept.

1. Ensure your deployment has replicas: 2.
2. In one terminal: kubectl get pods -w (watch continuously)
3. In another terminal: kubectl delete pod <one-of-your-pod-names>
4. Watch the first terminal: you will see the pod Terminating, then a new pod appearing in ContainerCreating, then Running — automatically.
5. Document what you saw in LEARNING_LOG.md: what happened, why, and what the practical implication is for running a production inference service.

✓ **Done when:** Learning Log entry accurately describes the self-healing cycle and explains why it matters for reliability.

💡 This is the core value proposition of Kubernetes. Your future inference services will crash sometimes — hardware faults, OOM kills, bugs. K8s self-healing means a single pod crash does not page an engineer at 3am.

PROJECT 3.4 WRITE A K8S TEST SCRIPT

Objective: Use the Kubernetes Python client to interact with your cluster programmatically — exactly what you will do as an SDET.

1. pip install kubernetes in a local virtualenv.
2. Write a Python script k8s_test.py that: loads your kubeconfig (from ~/.kube/config — minikube sets this up automatically), lists all pods in the default namespace, asserts that at least 2 pods with app=inference-sandbox are Running, prints a PASS or FAIL message with details.
3. Run it: python k8s_test.py
4. Scale your deployment to 1 replica, run it again — it should FAIL.
5. Scale back to 2, run again — PASS.

✓ **Done when:** Script correctly PASSes at replicas=2 and FAILs at replicas=1. No kubectl commands used inside the script — pure Python client.

💡 This script is a primitive version of what you will write in your Week 3 task in the internship guide. The kubernetes Python client is exactly what SDET teams use to write infrastructure tests that do not depend on shell commands.

Phase 3 Checkpoint

✅ **Before Moving to Phase 4** You should be able to: (1) explain what a Deployment does differently than just running docker run, (2) describe what a Service is for, (3) use kubectl describe and kubectl logs to debug a failing pod, (4) explain what a ConfigMap is and why it exists, (5) write a Python script that talks to K8s via the

client library. You do NOT need to understand everything about K8s — these five things are enough for day one.

PHASE 4 AI INFERENCE CONCEPTS

April 16 – April 30 (2 weeks) | *Hands-on inference benchmarking using free Colab GPUs + HuggingFace*

The Bridge Between Your Classes and Your Job

Your ML class teaches you how models are trained. Your job at NVIDIA is about how they run after training — inference. This phase connects those two worlds. You will load a real language model, measure how fast it generates text, and write tests that validate model quality under different precision settings. All of this runs on free Google Colab GPU instances.

> 🖥️ **Setup for This Phase** Go to colab.research.google.com. Create a new notebook. Runtime → Change runtime type → T4 GPU. Verify with: import torch; print(torch.cuda.is_available()) # Should print True. Save your notebooks to your inference-sandbox repo under a notebooks/ folder.

Core Concepts

The Two Phases of LLM Inference

Every time you ask a language model a question, two distinct phases happen:

```
# PREFILL PHASE
# Input: your full prompt ("What is the capital of France?")
# Process: the entire prompt is processed in ONE forward pass through the model
# Output: the KV Cache is populated; the first output token is generated
# Metric: TTFT (Time To First Token) — how long you wait before seeing any
response
# Characteristic: COMPUTE-BOUND — bottlenecked by raw GPU FLOPS

# DECODE PHASE
# Process: generates ONE token at a time, each requiring a full forward pass
# Each step reads the KV cache from all previous tokens
# Output: a stream of tokens until EOS (end of sequence) or max_tokens
# Metric: TPOT (Time Per Output Token) — latency between consecutive tokens
# Characteristic: MEMORY-BANDWIDTH-BOUND — bottlenecked by how fast VRAM can be
read

# Why this split matters for your job:
# Optimising TTFT = make the prefill faster (e.g., Flash Attention, chunked
prefill)
# Optimising TPOT = make decode faster (e.g., paged KV cache, speculative
decoding)
# These require DIFFERENT techniques — Dynamo disaggregates them for this exact
reason
```

Latency vs Throughput — Different Goals

These are the two fundamental performance metrics and they are often in tension:

```
# LATENCY = how long a single user waits for their response
# Measured as: TTFT, TPOT, total response time
# Low latency = fast experience for one user
# Minimised by: small batch sizes (less queuing), fast hardware

# THROUGHPUT = how many tokens the server generates per second across ALL users
# Measured as: tokens/second system-wide
# High throughput = server can handle many concurrent users
# Maximised by: large batch sizes (more work per GPU pass), continuous batching

# The tension:
# Batch size 1:   latency=10ms/token,  throughput=100 tokens/sec
# Batch size 32:  latency=40ms/token,  throughput=800 tokens/sec
#
# Serving systems set a target latency SLO and maximise throughput within it.
# Your job as an SDET is to verify both metrics stay within agreed bounds.
```

Quantisation — Making Models Faster and Smaller

Neural network weights are floating-point numbers. Using fewer bits per number reduces memory usage and speeds up computation — at some cost to accuracy.

Format	Bits	Bytes/Param	Relative Speed	Accuracy Loss	NVIDIA Support
FP32	32	4.0	1x (baseline)	None (baseline)	All GPUs
FP16	16	2.0	~2x	Negligible	All GPUs (Ampere+)
BF16	16	2.0	~2x	Negligible	Ampere+ (preferred)
INT8 (W8A8)	8	1.0	~3-4x	Small — must test	Turing+
FP8 (E4M3)	8	1.0	~4x	Very small on H100	H100+ only
INT4 (GPTQ)	4	0.5	~6x	Noticeable — test!	Ampere+

🔬 **The SDET Angle on Quantisation** Your job is not to implement quantisation — it is to test that the accuracy loss from quantisation is within an acceptable bound. The key metric is perplexity: a measure of how surprised the model is by text it should predict. Lower perplexity = better model. FP16 perplexity should be within ~0.5 of FP32. If it is not, the quantisation is broken or misconfigured.

KV Cache — The Memory Bottleneck

During the decode phase, every forward pass needs to "remember" the attention keys and values from all previous tokens. Storing these is the KV cache. For a 70B parameter model generating 4096 tokens for 32 concurrent users, the KV cache can consume tens of gigabytes of VRAM. Managing this is why Paged KV Cache (like TRT-LLM's implementation) and Dynamo's disaggregated architecture exist.

```
# KV cache size estimation (rough):
#
# kv_cache_bytes =
#     num_layers
#     x 2 (key + value)
```

```
#      x num_kv_heads
#      x head_dim
#      x seq_len
#      x bytes_per_element
#      x batch_size
#
# Example: LLaMA-3-8B, seq_len=4096, batch=32, FP16
# = 32 layers x 2 x 8 heads x 128 dim x 4096 tokens x 32 batch x 2 bytes
# ≈ 8.6 GB  — just for the KV cache, before weights (16GB for FP16)
#
# This is why H100 with 80GB HBM3 exists. 40GB A100 runs out fast.
```

Phase 4 Projects

📌 **All Projects in This Phase Run in Google Colab** Create a new notebook for each project. Save notebooks to notebooks/ in your repo. You do not need to pay for anything — the free T4 tier is sufficient for GPT-2.

PROJECT 4.1 BENCHMARK GPT-2 — TTFT AND THROUGHPUT

Objective: Measure real inference metrics on a real model — the core skill of AI infrastructure testing.

1. In a Colab notebook, install: pip install transformers torch
2. Load GPT-2 (117M parameters): from transformers import pipeline; gen = pipeline("text-generation", model="gpt2", device=0)
3. Write a function measure_ttft(prompt, generator) that uses time.perf_counter() to measure time from call to first token. (Hint: use generate() with streamer=TextIteratorStreamer and capture time to first chunk)
4. Write a function measure_throughput(prompts: list, generator) that generates all prompts sequentially, counts total tokens output, divides by total time.
5. Run both functions at batch sizes [1, 4, 8] using a list of varied prompts.
6. Plot TTFT and throughput on two side-by-side matplotlib charts. Save as benchmark_chart.png.
7. Add benchmark_chart.png to your repo via an MR.

✓ **Done when:** Charts exist in repo. Numbers are physically plausible: TTFT increases with batch size, throughput increases with batch size.

💡 If TTFT decreases as batch size increases, your measurement is wrong — you are measuring total time divided by batch, not individual request latency. TTFT should always get worse (larger) as you add more concurrent requests.

PROJECT 4.2 ADD HUGGINGFACE INFERENCE TO BENCHMARK.PY

Objective: Extend your throughline project to benchmark a real model — not just string reversal.

1. Add transformers and torch to requirements.txt.
2. Add a --model flag to benchmark.py (default: "gpt2").
3. Add a run_llm_benchmark(model_name, prompts) function that: loads the model, runs each prompt through it with max_new_tokens=50, measures TTFT and tokens/sec for each, returns a list of result dicts.
4. Update save_results() to handle both the old string-reversal results and new LLM benchmark results (add a "type" field to differentiate).
5. Update --self-test to skip the LLM test if no GPU is available (check torch.cuda.is_available()) so CI still passes without a GPU.
6. Rebuild your Docker image. Verify it still builds and --self-test passes.

✓ **Done when:** benchmark.py runs LLM benchmarks on GPU, falls back gracefully on CPU. Docker image builds clean. CI passes.

💡 The torch.cuda.is_available() guard is a real pattern used in NVIDIA's own test suites — tests skip gracefully when the required hardware is absent rather than failing with a confusing error.

PROJECT 4.3 QUANTISATION ACCURACY TEST

Objective: Write a real quality assertion — the core SDET contribution to AI infrastructure.

1. In Colab, load GPT-2 in FP32: model = AutoModelForCausalLM.from_pretrained("gpt2", torch_dtype=torch.float32)
2. Load GPT-2 in FP16: model_fp16 = AutoModelForCausalLM.from_pretrained("gpt2", torch_dtype=torch.float16)
3. Compute perplexity for both on a 200-sentence subset of WikiText-2 (available via datasets library: load_dataset("wikitext", "wikitext-2-raw-v1"))
4. Write an assertion: assert abs(ppl_fp16 - ppl_fp32) < 0.5, f"FP16 perplexity {ppl_fp16:.2f} too far from FP32 {ppl_fp32:.2f}"
5. Add this as a standalone test file tests/test_quantisation.py using pytest. The test should skip if no GPU is available (use pytest.mark.skipif).
6. Add a note in your Learning Log about what the perplexity delta you observed actually means for model output quality.

✓ **Done when:** pytest tests/test_quantisation.py passes. Assertion is correctly structured. You can explain what the perplexity numbers mean in plain English.

💡 Perplexity is the exponential of average negative log-likelihood. A perplexity of 30 means the model is "choosing between 30 equally likely tokens" at each position on average. Lower = better. FP16 should be within ~0.5 of FP32 for a well-behaved quantisation — that is your quality gate.

PROJECT 4.4 KV CACHE PRESSURE OBSERVATION

Objective: Directly observe the memory behaviour that motivates paged KV caches in TRT-LLM.

1. In Colab, use torch.cuda.memory_allocated() before and after generating text at increasing sequence lengths: 128, 256, 512, 1024 tokens.
2. Print the VRAM used at each step. Plot VRAM vs sequence length.
3. Write a paragraph in your Learning Log explaining: why VRAM grows with sequence length, what would happen at sequence length 32768 on a 16GB GPU, and what "paged KV cache" might do to address this.
4. (Do not just copy the explanation from earlier in this guide — reason it out yourself.)

✓ **Done when:** Plot exists. Learning Log explanation is accurate and in your own words.

💡 If VRAM does not grow linearly with sequence length, check that you are measuring the right thing — the KV cache accumulates across the decode steps, not just at generation start.

Phase 4 Checkpoint

✅ **Before Moving to Phase 5** You should be able to explain without notes: TTFT vs TPOT (what they measure, what affects them), the latency/throughput tradeoff at different batch sizes, what quantisation trades off and how perplexity measures that tradeoff, and why the KV cache is a memory bottleneck. You should also have real benchmark numbers you measured yourself to reference in interviews.

PHASE 5 TRT-LLM HANDS-ON

April 30 – May 14 (2 weeks) | *Build a real TRT-LLM engine and write a correctness regression test*

What TRT-LLM Is and Why It Exists

PyTorch is a research framework — flexible, readable, but not optimised for production inference throughput. TensorRT-LLM is NVIDIA's production-grade alternative: it takes a trained model, compiles it into a highly optimised GPU binary (an "engine"), and runs it with custom CUDA kernels for attention, KV cache management, and batching. A TRT-LLM engine for LLaMA-3-8B is typically 3–5x faster than the equivalent PyTorch model running naively. This is what serves billions of tokens per day in production.

GPU Access for This Phase

You need a real NVIDIA GPU for TRT-LLM. Two practical options:

Option	Cost	GPU	Best For	How to Access
Kaggle Notebooks	Free	T4 or P100	Getting started, smaller models	kaggle.com → New Notebook → GPU accelerator
Lambda Labs	~$1–2/hr	A100 or H100	Full TRT-LLM workflow, FP8 tests	lambdalabs.com → Cloud → On-demand

Start with Kaggle (free). If you need more power or the Kaggle environment is limiting, budget 2–3 hours on Lambda ($3–5 total). That is enough to complete all Phase 5 projects.

The TRT-LLM Build Pipeline

There are three steps to go from a trained model to a running TRT-LLM server. You will do all three:

```
# ─── Step 1: Pull the TRT-LLM Docker Container ──────────────────────────
# On Kaggle/Lambda, pull the official NGC container.
# This has everything pre-installed: TensorRT, TRT-LLM, CUDA, cuDNN.
#
docker pull nvcr.io/nvidia/tensorrt-llm/release:0.17.0
# OR on Kaggle, install directly:
pip install tensorrt-llm --extra-index-url https://pypi.nvidia.com

# ─── Step 2: Clone the TRT-LLM repo (for examples and conversion scripts) ────
git clone https://github.com/NVIDIA/TensorRT-LLM
cd TensorRT-LLM

# ─── Step 3: Convert HuggingFace weights to TRT-LLM format ──────────────────
python examples/llama/convert_checkpoint.py \
    --model_dir TinyLlama/TinyLlama-1.1B-Chat-v1.0 \   # HuggingFace model ID or
local path
    --output_dir /tmp/trtllm_checkpoint \
    --dtype float16
```

```
# ── Step 4: Build the TRT engine ──────────────────────────────────────
trtllm-build \
    --checkpoint_dir /tmp/trtllm_checkpoint \
    --output_dir /tmp/trtllm_engine \
    --max_batch_size 8 \
    --max_input_len 512 \
    --max_seq_len 1024 \
    --use_paged_context_fmha enable   # Enables Flash Attention variant for paged
KV cache

# ── Step 5: Run inference ─────────────────────────────────────────────
python examples/run.py \
    --engine_dir /tmp/trtllm_engine \
    --max_output_len 50 \
    --input_text "Explain neural network quantisation in simple terms"
```

📌 **Use TinyLlama for This Phase** TinyLlama-1.1B is the smallest supported model and builds in ~5 minutes on a T4. LLaMA-3-8B builds in ~20 minutes and requires more VRAM. Start with TinyLlama to learn the pipeline, then try a larger model if you have time and budget.

How TRT-LLM Differs from PyTorch Inference

Compiled vs Interpreted: PyTorch runs operations one by one at runtime. TRT-LLM compiles the entire computation graph into a single GPU binary ahead of time, fusing operations and selecting optimal kernels. Compilation is slow (minutes) but inference is much faster.

Fixed vs Dynamic Shapes: TRT engines are built for specific input shape ranges (min/opt/max). You define these at build time with --max_input_len and --max_batch_size. Requests outside these ranges will fail — your tests must validate the boundaries.

Engine Portability: An engine built on an A100 CANNOT run on a T4 or H100. Engines are compiled for a specific GPU architecture (compute capability). Your CI must enforce this — wrong architecture = silent wrong results, not a clean error.

No Python Overhead: The TRT-LLM runtime skips Python entirely during the hot path — the CUDA kernels run directly with no Python interpreter overhead. This is a major reason for the throughput improvement vs PyTorch.

Phase 5 Projects

PROJECT 5.1 BUILD YOUR FIRST TRT-LLM ENGINE

Objective: Go from a HuggingFace checkpoint to a running TRT-LLM engine — the central skill of Weeks 6–7.

1. On Kaggle (GPU), install tensorrt-llm from the NVIDIA PyPI index.
2. Clone the TensorRT-LLM repo.
3. Run convert_checkpoint.py for TinyLlama-1.1B-Chat (free on HuggingFace — no authentication needed).
4. Run trtllm-build with max_batch_size=4, max_input_len=256, max_seq_len=512.

5. Run examples/run.py with 3 different prompts. Record the outputs.
6. Time the build. Time inference at batch=1 and batch=4.
7. Save ALL commands you ran (exactly as run, with full output) to a file build_log.md in your repo under docs/. This is your reproducibility artifact.

✓ **Done when:** Engine files exist in output dir. examples/run.py produces coherent text output. build_log.md committed to your repo with exact, reproducible commands.

💡 If the build fails with a CUDA error, check that your TRT-LLM version matches your CUDA version. The NGC container handles this automatically — direct pip install is more fragile. If stuck, switch to the Docker container approach.

PROJECT 5.2 CORRECTNESS REGRESSION TEST

Objective: Write the test that validates a TRT-LLM engine produces outputs matching the HuggingFace reference.

1. Write tests/test_trtllm_correctness.py.
2. Define a list of 10 prompts. For EACH prompt: run with HuggingFace TinyLlama (greedy, temperature=0), run with TRT-LLM engine (greedy, same settings), compare output tokens (they should match exactly for greedy decoding).
3. Assert that at least 9/10 prompts produce matching output. Log mismatches with the full prompt, expected output, and actual output.
4. Add a pytest.mark.gpu marker. Add skip logic if no GPU is detected.
5. Write a short section in your Learning Log: why would a mismatch occur even with greedy decoding and identical settings?

✓ **Done when:** Test runs and passes at ≥9/10. Mismatches (if any) are logged with enough detail to diagnose. You can explain potential sources of mismatch.

💡 Common sources of mismatch: different tokeniser settings (add_special_tokens), sampling parameters not set identically (make sure both use do_sample=False, temperature=1.0 effectively disabled), or numerical precision differences causing slightly different token probabilities at the boundary.

PROJECT 5.3 PERFORMANCE BENCHMARK HARNESS

Objective: Integrate TRT-LLM benchmarking into your throughline benchmark.py.

1. Add a --engine flag to benchmark.py: if provided, run benchmarks against a TRT-LLM engine instead of a HuggingFace model.
2. Measure TTFT and tokens/sec at batch sizes [1, 2, 4].
3. Save results to your Redis store (via Docker Compose) using the same save_results() function from Phase 1.
4. Write a JSON file golden_baseline.json containing your measured numbers (e.g., {"batch_1_tps": 45.2, "batch_4_tps": 120.7}).
5. Write a test tests/test_perf_regression.py that loads golden_baseline.json and asserts current measurements are within 15% of baseline.
6. Deliberately degrade performance (add time.sleep(0.1) in the inference loop) and verify the test catches it.

✓ **Done when:** Regression test correctly fails when performance degrades. benchmark.py supports both --model (HuggingFace) and --engine (TRT-LLM) modes.

💡 The 15% tolerance exists because GPU performance has natural variance (thermal throttling, memory contention, etc.). Too tight a tolerance causes flaky tests. Too loose and you miss real regressions. 10–20% is the standard range for ML performance gates.

PROJECT 5.4 DOCUMENT YOUR END-TO-END PROJECT

Objective: Write a professional README that ties all six phases together.

1. Create README.md in your repo root.
2. Sections: Overview (what is inference-sandbox), Setup (how to run locally), Architecture (Docker, Compose, K8s, how it evolved), Running Tests (pytest commands), TRT-LLM (how to build an engine and use it with this project), What I Learned (honest reflection on the journey from Phase 1 to Phase 5).
3. Anyone who reads this README should be able to get the project running from scratch in < 30 minutes.
4. This is what you show your NVIDIA manager on day one.

✓ **Done when:** README is complete, accurate, and readable. Setup instructions actually work when followed on a fresh machine.

💡 Write it as if a new intern (not you) needs to use this project. Every instruction you skip because "it is obvious" will confuse that person. Clarity in documentation is a professional skill NVIDIA actively values.

Phase 5 Checkpoint

✅ **Before Moving to Phase 6** You have: (1) a TRT-LLM engine you built yourself, (2) a correctness regression test that compares it to a HuggingFace reference, (3) a performance benchmark harness with a golden baseline, (4) a professional README. This is more concrete TRT-LLM experience than most SDET interns arrive with. Phase 6 is intentionally lighter — you have earned it.

PHASE 6 NEMO + DYNAMO ORIENTATION

May 14 – May 18 (4 days) | *Build vocabulary and architecture understanding — you will go deep on these week 8–11*

The Goal of This Phase

NeMo and Dynamo are Weeks 8–11 material in your internship guide — your team will teach you these in depth. The goal here is NOT to master them. It is to arrive knowing what they are, why they exist, and where they fit in the stack — so you spend your first week asking smart questions instead of basic ones.

> 📌 **No GPU Needed This Phase** You are reading and reasoning, not running code. The deliverable is a well-organised notes document, not working software.

NeMo — What to Understand

NVIDIA NeMo is the framework that produces the model checkpoints that TRT-LLM compiles. Understanding NeMo means understanding where the model you are testing comes from.

NeMo vs HuggingFace: Both let you train and fine-tune transformers. NeMo adds Megatron-LM for multi-node tensor/pipeline parallelism — essential for 70B+ parameter models. HuggingFace is simpler but does not scale to NVIDIA's training cluster sizes.

SFT (Supervised Fine-Tuning): Taking a pre-trained base model and training it on instruction-following examples ({input: "...", output: "..."} pairs). This is how base models become assistants. NeMo is what NVIDIA uses to run SFT on internal datasets.

LoRA (Low-Rank Adaptation): A fine-tuning technique that adds small trainable adapter matrices to a frozen model. Only 0.1–1% of parameters are trained, reducing compute 10–100x vs full fine-tuning. NeMo supports LoRA natively. You will test LoRA-adapted checkpoints.

NeMo Evaluator: Runs standardised benchmarks (MMLU, HellaSwag, HumanEval) on a model checkpoint. Produces pass/fail results against defined thresholds. Your team will use these as quality gates before promoting a model to inference serving.

NeMo Curator: Data pipeline for web-scale training datasets: deduplication, language identification, quality filtering, PII removal. Data quality = model quality. You will write tests that validate the curator does not remove too much or leave PII in the training set.

Dynamo — What to Understand

Dynamo is what runs TRT-LLM at production scale across multiple nodes. Understanding Dynamo means understanding the infrastructure your tests run on.

The Problem Dynamo Solves: A single TRT-LLM server handles one node. Production traffic for a 70B model requires dozens of nodes. Dynamo provides the orchestration layer: routing, load balancing, failover, and scaling across the entire fleet.

Disaggregated Prefill/Decode: The most important Dynamo concept. Prefill (processing the prompt) is compute-bound. Decode (generating tokens) is memory-bandwidth-bound. Dynamo routes these to DIFFERENT GPU pools optimised for each workload type — maximising efficiency.

NIXL (NVIDIA Inference Xfer Library): When prefill finishes on one GPU node, the KV cache must move to the decode node instantly. NIXL does this via NVLink/RDMA in microseconds. Your latency tests will assert this transfer time is within SLO.

Why This Matters for Your Tests: Disaggregation introduces distributed systems failure modes: what if the KV transfer fails? What if the decode worker crashes mid-generation? Weeks 8–9 of your internship guide are about testing exactly these scenarios.

Phase 6 Project

PROJECT 6.1 ARCHITECTURE NOTES DOCUMENT

Objective: Produce the document that proves you understand the full stack before day one.

1. Create docs/architecture_notes.md in your inference-sandbox repo.
2. Section 1 — The Stack: Draw an ASCII diagram showing how NeMo, TRT-LLM, Dynamo, Kubernetes, and Docker relate to each other. Label which layer each technology lives in (training, compilation, serving, orchestration).
3. Section 2 — Data Flow: Trace the path of a user prompt from HTTP request to generated token in a full Dynamo deployment. Every step, in order.
4. Section 3 — Where inference-sandbox Fits: Explain how your project (currently testing TRT-LLM directly) would need to change to test a Dynamo-served endpoint instead.
5. Section 4 — Open Questions: Write down 5 things you do NOT understand yet about NeMo or Dynamo. These become your questions for week 1 at NVIDIA.
6. Commit to repo via MR. This document is interview and day-one gold.

✓ **Done when:** Document exists, diagrams are accurate, data flow trace is correct, open questions show genuine curiosity and self-awareness.

💡 The open questions section matters as much as the rest. Walking into your first 1:1 with 5 specific, well-formed questions signals exactly the right level of preparation. Your manager will remember it.

Phase 6 Checkpoint — And Your Final State

✅ **What You Have Built by May 18** inference-sandbox is a GitLab repo with: (1) a benchmark.py that benchmarks both HuggingFace models and TRT-LLM engines, (2) a multi-stage Dockerfile and docker-compose.yml with Redis integration, (3) Kubernetes deployment + service + configmap manifests, (4) a correctness regression test and performance benchmark harness, (5) 10 weeks of Learning Log entries, (6) a professional README and architecture notes doc. That is a portfolio piece, not just prep work.

Appendix A: Your Complete Repo Structure

By the end of Phase 6, inference-sandbox should look like this:

```
inference-sandbox/
├── benchmark.py              # Core benchmarking script (grows through all
phases)
├── k8s_test.py               # Kubernetes Python client test (Phase 3)
├── requirements.txt          # Python dependencies
├── Dockerfile                # Multi-stage build (Phase 2)
├── docker-compose.yml        # App + Redis stack (Phase 2)
├── .gitlab-ci.yml            # CI pipeline (Phase 1 → grows each phase)
├── .gitignore                # Excludes *.jsonl, __pycache__, .env, etc.
├── README.md                 # Professional setup + usage guide (Phase 5)
├── LEARNING_LOG.md           # Weekly reflections (all phases)
├── golden_baseline.json      # TRT-LLM performance baseline (Phase 5)
├── k8s/
│   ├── deployment.yaml       # Kubernetes Deployment (Phase 3)
│   ├── service.yaml          # Kubernetes Service (Phase 3)
│   └── configmap.yaml        # Kubernetes ConfigMap (Phase 3)
├── tests/
│   ├── test_quantisation.py  # FP32 vs FP16 perplexity test (Phase 4)
│   ├── test_trtllm_correctness.py  # TRT-LLM vs HuggingFace (Phase 5)
│   └── test_perf_regression.py     # Throughput regression gate (Phase 5)
├── notebooks/
│   ├── phase4_benchmark.ipynb      # TTFT + throughput (Phase 4)
│   ├── phase4_quantisation.ipynb   # Perplexity comparison (Phase 4)
│   └── phase4_kv_cache.ipynb       # VRAM observation (Phase 4)
└── docs/
    ├── build_log.md                # TRT-LLM exact build commands (Phase 5)
    └── architecture_notes.md       # NeMo + Dynamo diagrams (Phase 6)
```

Appendix B: Free Resources

Tools to Install on Day 1 of Phase 1

Git: git-scm.com/downloads

Docker Desktop: docs.docker.com/get-docker

minikube: minikube.sigs.k8s.io/docs/start (install in Phase 3)

kubectl: kubernetes.io/docs/tasks/tools (install with minikube)

Python 3.11: python.org/downloads

VS Code: code.visualstudio.com (recommended editor)

Free GPU Platforms

Platform	GPU	Free Hours/Week	Best For	URL
Google Colab	T4	~12–15 hrs	Phase 4 notebooks	colab.research.google.com

Kaggle Notebooks	T4 or P100	30 hrs	Phase 5 TRT-LLM	kaggle.com/code
Lambda Labs	A100 / H100	Pay-per-hour ($1–4/hr)	Phase 5 if Kaggle insufficient	lambdalabs.com

Key Repositories to Explore (Read, Not Run)

TensorRT-LLM: github.com/NVIDIA/TensorRT-LLM — study tests/ and examples/ directories

NeMo: github.com/NVIDIA/NeMo — study tutorials/ and examples/nlp/

Dynamo: github.com/ai-dynamo/dynamo — read the architecture docs in docs/

TensorRT: github.com/NVIDIA/TensorRT — read samples/ for core TRT concepts

Model Optimizer: github.com/NVIDIA/Model-Optimizer — quantisation pipeline used with TRT-LLM

Appendix C: Weekly Learning Log Template

Create one entry per week in LEARNING_LOG.md. Use this format every time. Consistency matters more than length.

```
# Phase N — Week Starting [DATE]

## What I Built This Week
<!-- Describe the concrete output: what exists now that did not before? -->

## What I Learned
<!-- 3-5 key concepts, in your own words. Not copied. If you can't explain it,
you don't know it yet. -->

## What Confused Me (and how I resolved it)
<!-- Specific confusion + specific resolution. "I was confused about X, then I
did Y and it clicked." -->

## What Surprised Me
<!-- One thing that was different from what you expected. Good surprises and bad
ones. -->

## Open Questions
<!-- Things you still don't understand. These become your questions for your
internship manager. -->

## Checkpoint Status
<!-- Copy the checkpoint criteria from the phase. Mark each: [x] Done / [ ] Not
yet -->
```

Appendix D: Glossary of Terms You Will Hear Week One

Autoregressive Decoding: The process of generating text one token at a time, where each token depends on all previous tokens. The core loop of LLM inference.

BF16: Brain Float 16. 16-bit floating point format with the same exponent range as FP32, making it numerically stable for training. Preferred over FP16 on Ampere+ GPUs.

Continuous Batching: A serving strategy where new requests join a running batch as soon as a slot opens, without waiting for all current requests to finish. Massively improves throughput.

cuDNN: NVIDIA's deep learning primitives library. Provides optimised CUDA implementations of convolution, attention, and other DL operations. Bundled in CUDA runtime images.

Device Plugin: A Kubernetes component (nvidia/k8s-device-plugin) that exposes GPU resources to the K8s scheduler, enabling pods to request GPUs via nvidia.com/gpu: 1.

EOS Token: End of Sequence. A special token the model generates to signal it has finished its response. LLM servers stop generating when they see this.

FP8: NVIDIA H100-native 8-bit float format. Two variants: E4M3 (better range) and E5M2 (better precision). TRT-LLM's default quantisation on H100, ~2x faster than FP16.

Greedy Decoding: Always selecting the highest-probability next token. Deterministic — the same prompt always produces the same output. Used as the reference for correctness tests.

HBM3: High Bandwidth Memory 3. The VRAM type on H100 GPUs. 80GB capacity, 3.35 TB/s bandwidth. The decode phase is bottlenecked by this bandwidth.

KV Cache: Key-Value cache storing attention tensors from previous tokens during autoregressive decode. Avoids recomputing attention over the full sequence at each step. VRAM-intensive.

LoRA: Low-Rank Adaptation. A fine-tuning technique that adds small trainable adapter matrices to frozen model weights. Only trains ~0.1% of parameters.

Megatron-LM: NVIDIA's internal framework for training trillion-parameter models using tensor, pipeline, and sequence parallelism across thousands of GPUs. NeMo wraps it.

NGC: NVIDIA GPU Cloud. NVIDIA's registry of pre-built containers, pre-trained models, and SDKs. nvcr.io is the container registry URL.

NVLink: NVIDIA's proprietary inter-GPU interconnect. On H100 NVL systems: 900 GB/s bidirectional bandwidth between GPUs. Used by Dynamo's NIXL for KV cache transfers.

ONNX: Open Neural Network Exchange. Intermediate format for exporting models from PyTorch/TF for use with TensorRT or other inference runtimes.

Perplexity: Measure of language model prediction quality. Exponential of average negative log-likelihood. Lower = better. Used to compare model quality across quantisation levels.

Pipeline Parallelism: Splitting model layers across multiple GPUs in sequence (GPU 0 runs layers 1–16, GPU 1 runs layers 17–32). Used for very deep models.

Prefill: The phase of LLM inference where the full input prompt is processed in one forward pass and the KV cache is populated. Determines TTFT.

SLO: Service Level Objective. A target value for a metric (e.g., p99 TTFT < 200ms). Your test suite enforces SLOs — that is the SDET's job.

Speculative Decoding: Use a small draft model to propose K tokens, then verify them in one pass of the large model. Reduces TPOT at the cost of running an extra small model.

Tensor Parallelism: Splitting a matrix multiplication across multiple GPUs (each computes a shard). Requires an all-reduce communication step after each layer.

TPOT: Time Per Output Token. Average latency between consecutive generated tokens during the decode phase. Bottlenecked by memory bandwidth.

TTFT: Time To First Token. Latency from request arrival to when the first output token is generated. Bottlenecked by the prefill compute phase.

10 weeks from now,

you will have built something real.

Not read about it. Not watched a tutorial about it.

Built it, broken it, debugged it, and shipped it.

That is the difference between a resume line

and something you actually know.

See you on May 18.

— NVIDIA Deep Learning Infrastructure